



LANGAGES À OBJET AVANCÉS

Baba Is You

Gabriel Choucroun

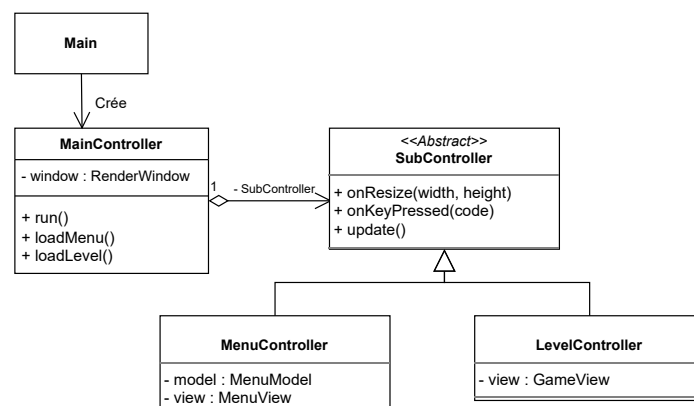
Anthony Fernandes

Binôme n°89

I. Architecture globale

Nous avons choisi le MVC (Model-View-Controller) comme patron principal de notre projet. Celui-ci nous permet de gérer la boucle principale de notre jeu, dans le `MainController`. Ce `MainController` est la classe principale d'un strategy pattern dont `main` est le client. Les stratégies possibles sont les sous-classes de `SubController`, il y a un `SubController` par menu : un pour le menu principal, et un pour le jeu. Le `MainController` attrape les événements de la fenêtre et les transmet au `SubController` actuellement en charge via ses fonctions `onResized` et `onKeyPressed`. Ceci nous permet de fluidifier les changements de vue, lors des transitions entre menu et niveau.

Le changement de stratégie consiste à supprimer le `SubController` actuel et à le remplacer par un nouvel objet. Cette modélisation permet une gestion autonome de la vue et du modèle, par exemple le `LevelController` gère tout seul son affichage (`GameView`) et son modèle (`Game`).



```
1 void MainController::loadMenu() {
2     delete subController;
3     subController = new MenuController{*this, window};
4 }
5 void MainController::loadLevel(std::string path) {
6     delete subController;
7     subController = new LevelController{*this, window, path};
8 }
```

Fig. 1. – Le strategy pattern et les fonctions de changement de stratégie

Nous avons choisi de stocker les niveaux dans des fichiers texte légers (.txt), ces fichiers sont organisés comme des csv. La première ligne correspond à la taille du niveau (nombre de colonne et de ligne), chaque ligne suivante est une ligne de la grille dont les cases sont séparées par des virgules. Dans une case les entités sont représentées par leur nom, éventuellement précédé d'une esperluette pour les références. Le `MenuModel` fait correspondre l'affichage du menu aux niveaux proposés, et permet d'obtenir le chemin vers le niveau sélectionné pour que `Game` puisse charger le fichier.

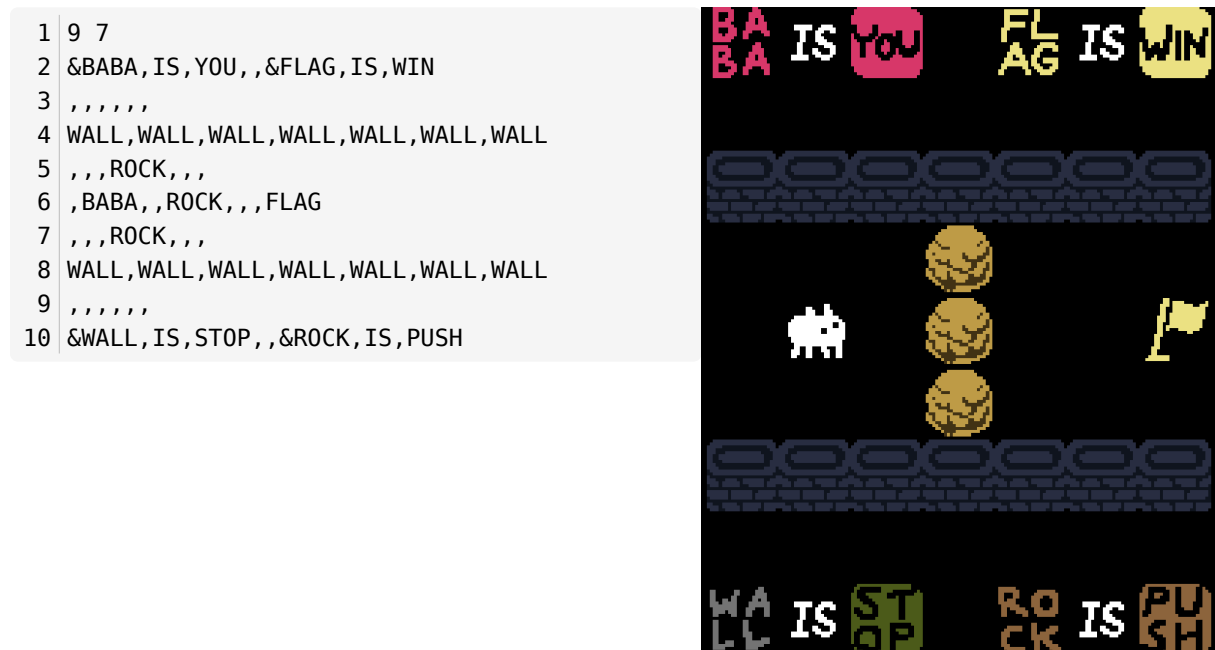


Fig. 2. – Un fichier de niveau et son affichage

II. Modèle

Nous avons choisi de représenter de façon uniforme tout ce qui se trouve sur le terrain de jeu par des objets de type `Entity` ; que ce soit les personnages, les éléments de décors ou le texte. Une entité possède un ensemble de propriétés qui influencent la façon dont elle interagit avec les autres entités, cet ensemble est modifié le long de la partie pour refléter l'état actuel des blocs de texte. Via l'héritage, on distingue alors deux grandes classes d'entités :

- D'une part les `BasicEntity` possédant chacune leur ensemble de propriétés, et qui n'ont aucune caractéristique particulière autre qu'exister. Elles représentent tout ce qui est sur la grille sauf le texte, par exemple Baba ou un rocher.
- D'autre part les `TextEntity` qui représentent tous les blocs de texte. Ces entités partagent toutes un même ensemble de règles comme variable statique de la classe `TextEntity`, c'est ce qui nous permet d'implémenter le bloc de règle TEXT et de nous assurer qu'il est toujours possible de pousser du texte. Cette classe a trois sous classes :
 - `RefEntity` pour les blocs de référent à d'autres entités (ils apparaissent à gauche des règles). Ces blocs ont une référence `Entity` &ref.
 - `Operator` pour les opérations que l'on intercale entre les autres blocs de texte. Il n'y a qu'un seul opérateur : IS ; mais si nous ajoutions les blocs AND, HAS ou NOT, ils seraient de type `Operator`.
 - `Property` pour les blocs représentant les règles comme YOU ou PUSH. Cette classe est particulière puisqu'elle sert à la fois à être placée sur la grille comme élément de jeu mais également à être stockée dans les ensembles de règles des autres entités. Cette classe possède deux event handler

abstrait `onEnter` et `onStay` pour permettre l'application des règles (voir plus loin), chaque propriété redéfinit ces handlers via une classe anonyme étendant `Property`.

Puisque toutes les entités sur la grille représentant un objet fixé sont parfaitement identiques (par exemple tous les Babas présents sont en réalité exactement le même), nous avons décidé d'opter pour ce qui s'apparente à un pattern singleton. Les constructeurs de copie et opérateurs d'assignation des entités ont été supprimés et les constructeurs sont privés. Ainsi les seules entités pouvant exister sont déclarées comme variables globales et sont créées sur la pile au lancement du programme, durant tout le reste de l'exécution, seule des références et des pointeurs vers ces entités globales sont passés.

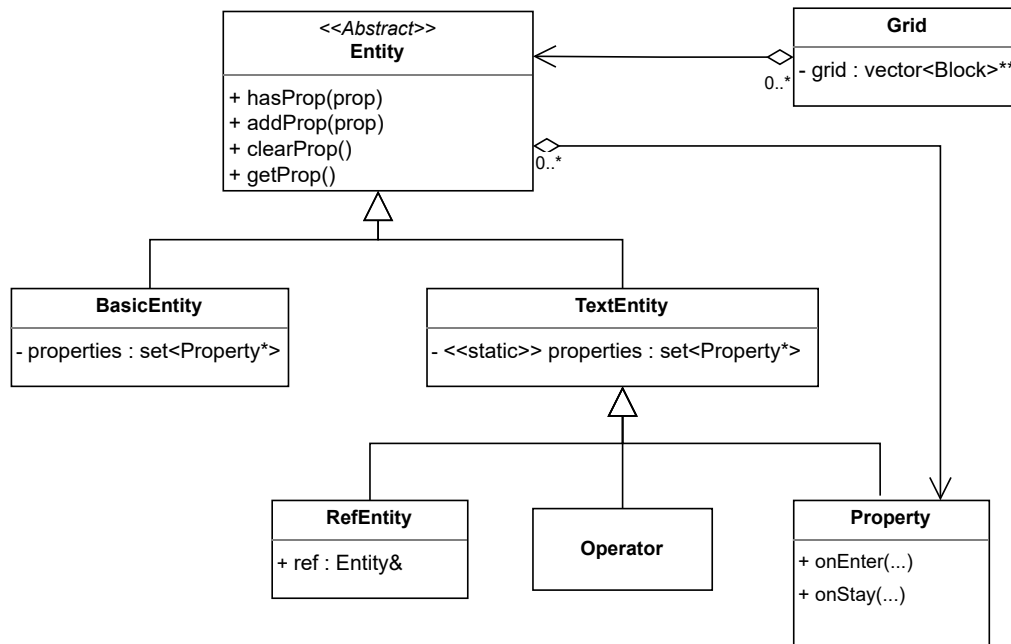


Fig. 3. – Diagramme de classe des entités

L'état global de la grille est stocké dans une classe `Grid` qui n'est rien d'autre qu'une encapsulation pour une table bidimensionnelle de vecteurs de `Block` (un `Block` est pointeur d'entité muni d'une direction dans laquelle elle regarde). Nous avons muni la grille de quelques opérations utilitaires pour pouvoir la manipuler plus facilement, notamment des opérateurs `()` et `[]` nous permettant d'accéder directement à une case via ses coordonnées ainsi que d'écrire `grid[prop]` pour récupérer toutes les entités ayant la propriété `prop` ou encore `grid[ptr]` pour récupérer toutes les positions où se trouve l'entité pointée par `ptr`.

Les modifications de la grille ne sont pas directement appliqués mais passent d'abord sous la forme d'une `Action`. Une action est faite de deux listes : une d'entités à ajouter sur la grille et une d'entités à supprimer. Quand une action est utilisée pour modifier la grille, celle-ci est enregistrée dans l'historique (classe `History`) pour permettre son annulation.

Les actions sont obtenues par une interrogation du `RuleManager`. Lorsqu'une entité tente d'entrer sur une case, le `RuleManager` regarde chaque propriété de chaque entité sur case d'arrivée et lui demande quoi faire via son event handler `onEnter` qui renvoie une action. Si l'entité est autorisée à se déplacer (les propriétés `STOP` et `SHUT` empêchent le déplacement), celle-ci est appliquée. Une fois toutes les entités déplacées, un second passage est fait, cette fois en appelant de handler `onStay` pour faire interagir entre elles les entités se trouvant maintenant sur la même case (par exemple la propriété `WIN` déclenche la victoire si une propriété `YOU` se trouve sur la même case).

La classe principale du modèle, `Game`, n'est qu'une composition de `Grid`, `History` et `RuleManager` et les fait fonctionner ensemble via sa fonction `move`.

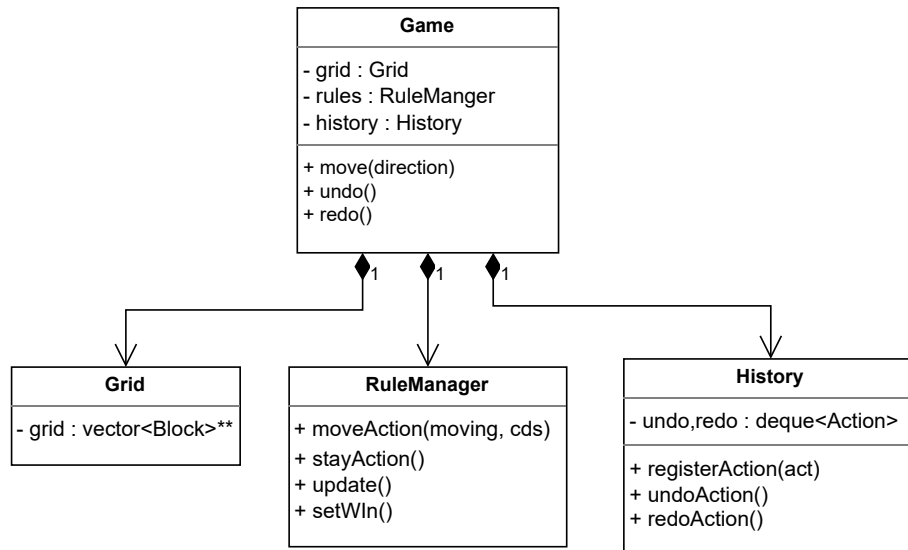


Fig. 4. – Diagramme de classe des entités

III. Vue

Comme pour le modèle, nous avons choisi de représenter de façon uniforme tout ce qui est affiché sur le terrain de jeu, par des objets de type `Animation`. Toute la logique de notre affichage utilise des Fly-weight Patterns. D'une part, les spritesheet (reprises du jeu original) ne sont générés qu'une seule fois, grâce au cache de la classe `SpritesheetFactory`. De même pour chaque entités, l'objet de l'animation correspondante n'est créé qu'une seule fois, grâce au cache de la classe `AnimationFactory`.

```

1  const Animation &AnimationFactory::operator[](const Entity *e) const {
2      Animation *anim = cache[e];
3      if (anim == nullptr) anim = makeAnim(e);
4      return *anim;
5  };
6
7  Animation *makeAnim(const Entity *e) {
8      if (e == &BasicEntity::BABA) return makeCharacter({576, 1});
9      if (e == &BasicEntity::FLAG) return makeObject({351, 226});
10     if (e == &RefEntity::NBABA) return makeText({551, 1}, "characters.png");
11     if (e == &RefEntity::NFLAG) return makeText({326, 226}, "objects.png");
12     if (e == &Property::YOU) return makeText({351, 226});
13     if (e == &Operator::IS) return makeText({226, 76});
14     // ...
15     throw std::logic_error("Sprite not found"); // Fallback
16 }

```

Fig. 5. – Le système de cache de la classe `AnimationFactory`

Un objet `Animation` est créé avec les positions, indiquant là où trouver l'entité dans la spritesheet. De la même manière que dans notre modèle, on distingue nos différentes animations, ici via un héritage multiple qui permet d'obtenir des animations de plus en plus complexe:

- Les `StaticSprite` correspondent aux entités qui n'ont pas d'animation dans l'affichage, par exemple nos blocs de texte.

- Les `DirectedSprite`, héritant de `StaticSprite`, représentent les entités qui s'animent en changeant de direction. Nous n'en n'avons pas intégré, mais nous pourrions ajouter par exemple des crânes qui tournent.
- Les `AnimatedSprite` représentent les entités qui s'animent en changeant de sprite, mais sans prendre en compte la direction. Nous n'en n'avons pas intégré, mais nous pourrions ajouter par exemple des bulles de décoration.
- Les `FullAnimation`, héritant de `DirectedSprite` et de `AnimatedSprite`, représentent les entités comme Baba et Keke qui s'animent en changeant de sprite à chaque mouvement, et en s'adaptant à la direction.

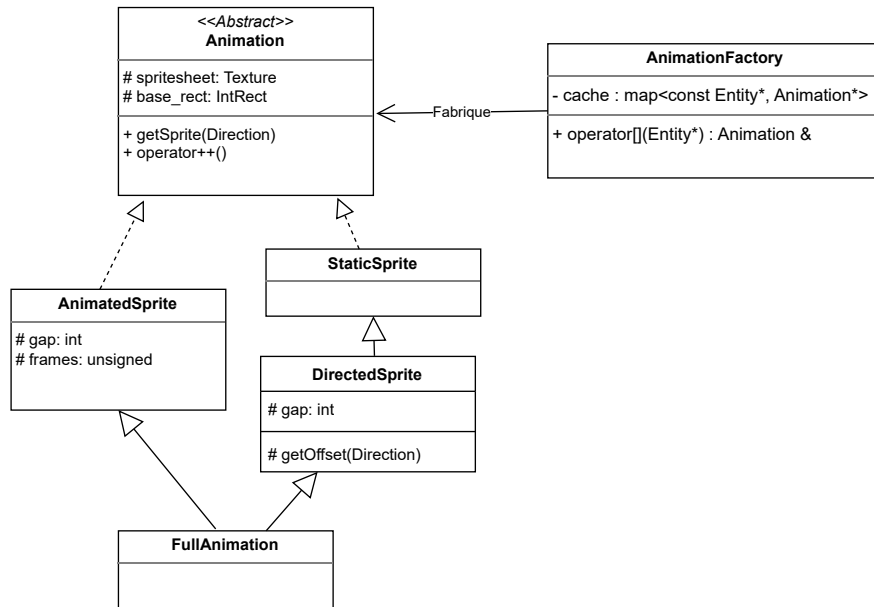


Fig. 6. – Diagramme de classe des animations

Les animations complexes utilisent leurs attributs `gap` et `frame` pour retrouver toutes les images de l'animation par décalage de la position initiale. La classe principale de la vue, `GameView`, obtient les animations par des appels à son `AnimationFactory`, pour produire les affichages de la grille. On actualise la vue via ses fonctions `advanceAnimation()` et `draw()`, cette dernière est héritée de la classe `sf::Drawable`.

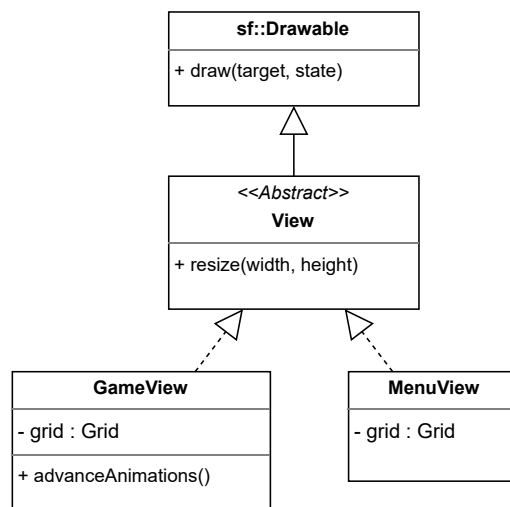


Fig. 7. – Diagramme de classe des vues

IV. Extensions et améliorations possibles

Le projet répond à toutes les parties obligatoires du sujet. De plus, nous avons traité les fonctionnalités supplémentaires suivantes:

- ajouter KEY et DOOR
- gérer les règles de la forme « WALL IS BABA »

Grâce à la modularité et à la gestion réfléchie de la mémoire dans notre code, nous n'avons rencontré qu'un seul problème, relativement secondaire : lorsqu'un objet possède OPEN et entre en interaction avec un objet CLOSE, l'objet ayant initié l'action ne se déplace pas sur la case de l'objet OPEN maintenant disparu.

Concernant les autres extensions proposées :

- Nous trouvions intéressent les extensions PULL et AND. L'extension AND nous aurait nécessité une lecture des règles plus intelligentes mais le gros de la structure est déjà là. PULL, quant à elle, nécessite de changer complètement notre système d'action actuel.
- Nous n'avons pas considéré l'extension « A IS LEFT » car casserait le principe de séparation entre modèle et vue du MVC.
- La sauvegarde, suite de niveaux et l'éditeur n'était, d'après nous, pas l'enjeu principal du projet et représentaient un investissement en temps conséquent que nous n'aurions pas pu utiliser sur le jeu.
- Finalement nous n'avons pas fait d'écran de victoire, ceci n'impacte pas le gameplay, et ne prendrait que quelques ligne de code à implémenter.

Annexe A. Diagrammes complets

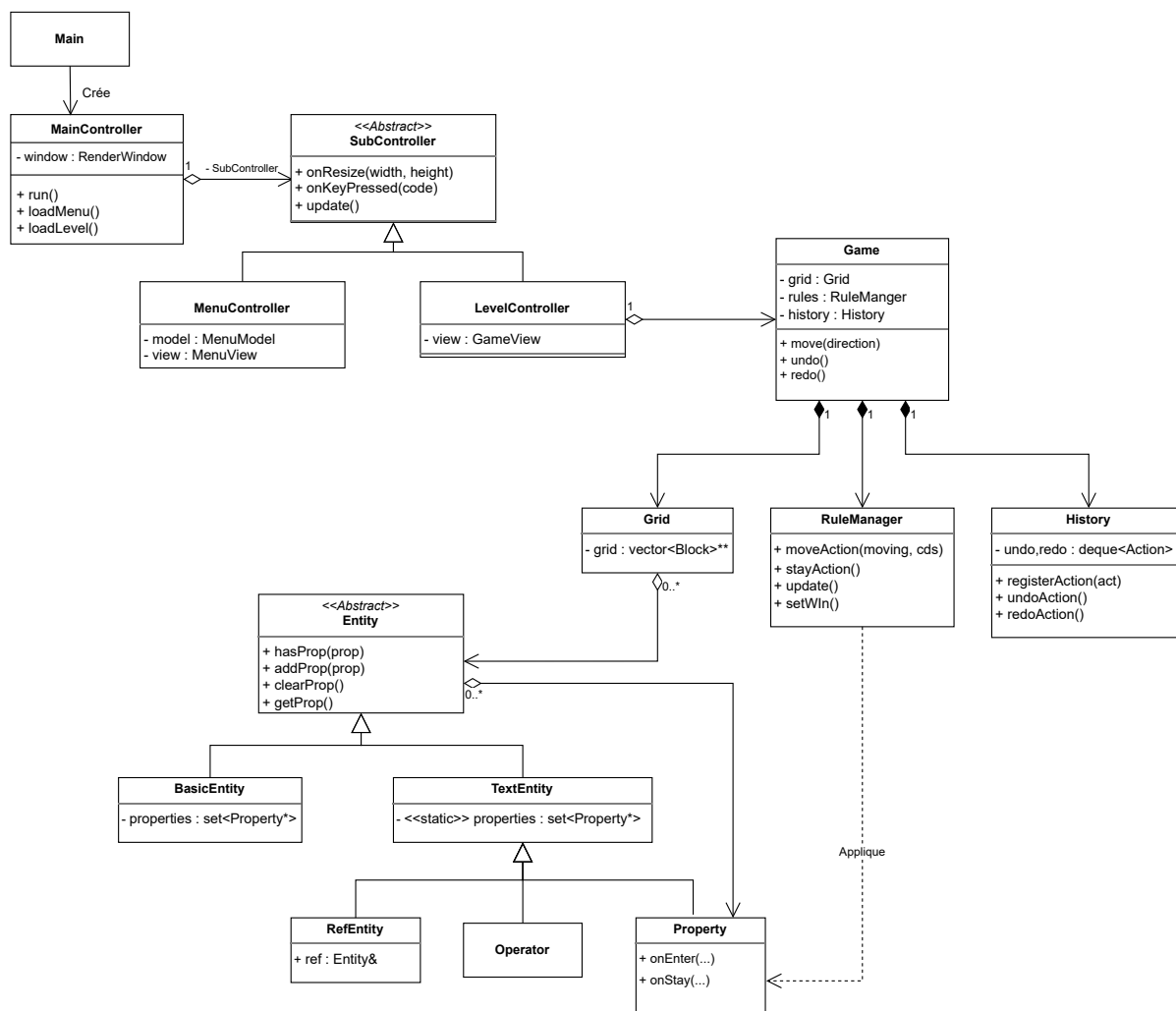


Fig. 8. – Diagramme complet du modèle et du contrôleur

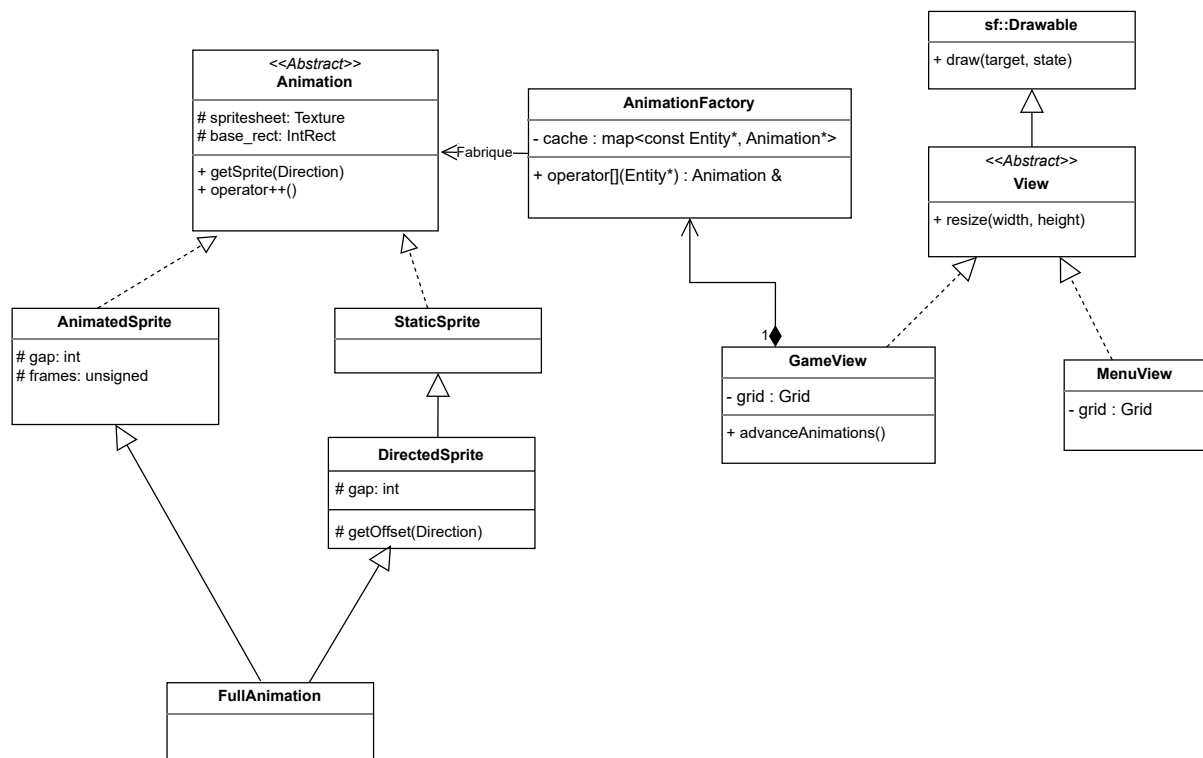


Fig. 9. – Diagramme complet de la vue